

Chapter 2

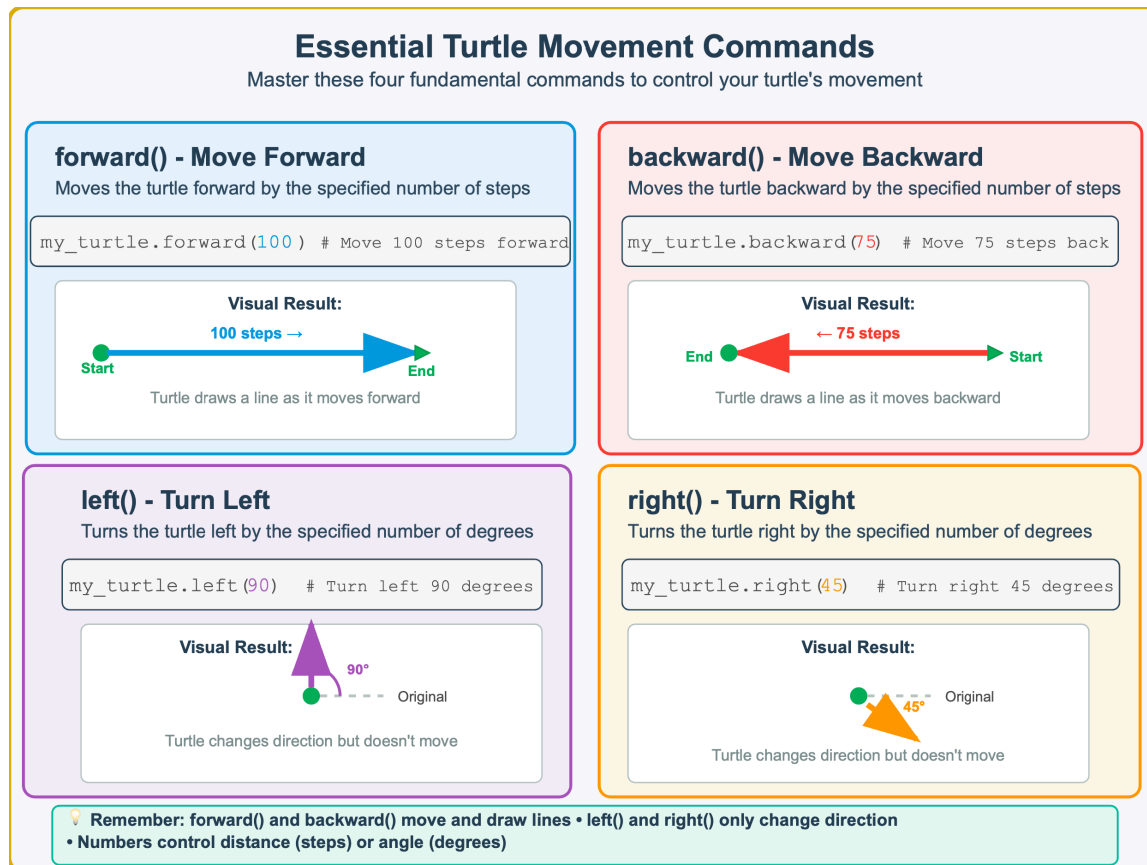


Figure 2.1 — Essential Turtle Movement Commands — Complete Reference Guide. Master these four fundamental commands—forward(), backward(), left(), and right()—and you'll have the building blocks for creating any shape or pattern. The numbers in parentheses control distance (in steps) or angle (in degrees), giving you precise control over your turtle's movement

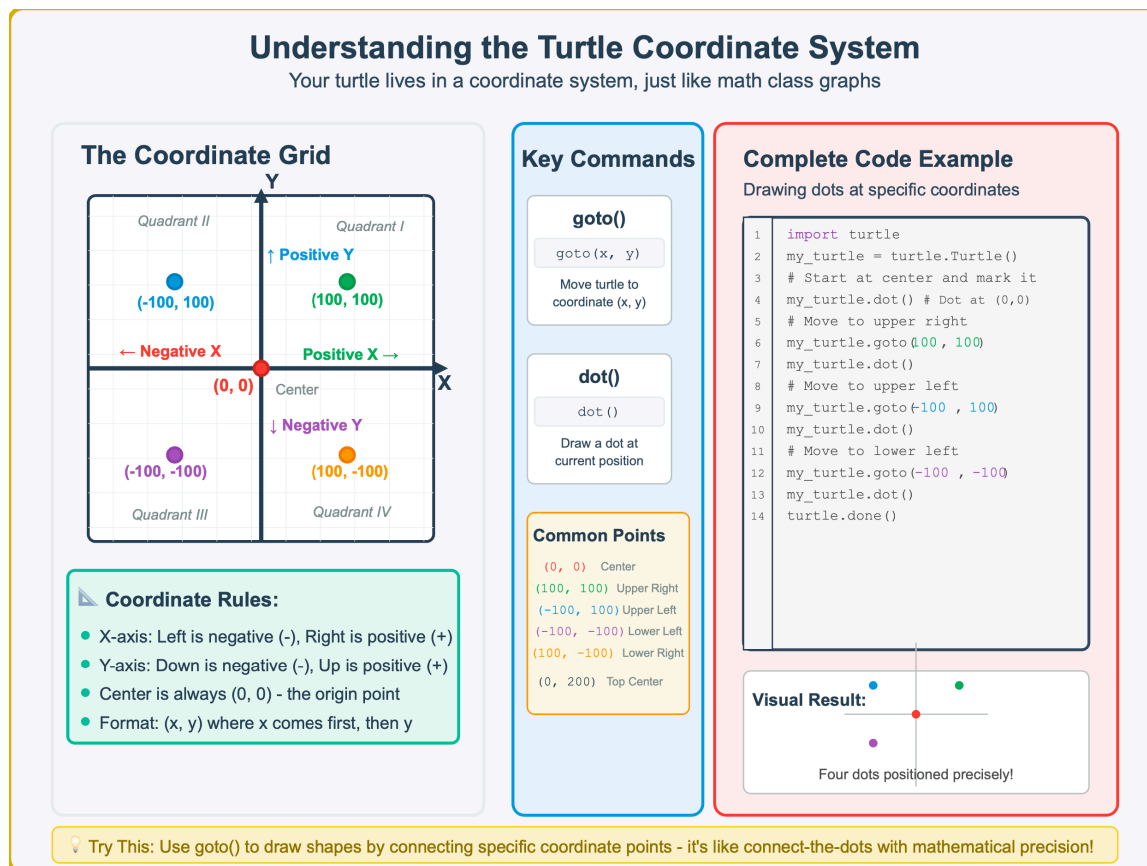


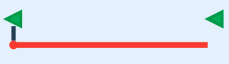
Figure 2.2 — Understanding the Turtle Coordinate System — From Graph to Code. The coordinate system you learned in math class becomes your turtle’s GPS system. Every position on the screen has an (x, y) address, and the goto() command lets you send your turtle to any location instantly. This mathematical precision transforms random drawing into planned artwork.

Mastering Pen Control

Your turtle carries a pen that you can control completely

Pen Control Basics

Pen Down - Drawing Mode



Turtle draws a line as it moves

`pendown()`

Pen Up - Moving Mode



Turtle moves without drawing

`penup()`

Basic Example:

```
my_turtle.penup()
my_turtle.goto(50, 50)
my_turtle.pendown()
my_turtle.forward(100)
```

Move without drawing, then draw

Command Reference

Pen Control

`pendown()`

`penup()`

Colors

`color("red")`

`color("blue")`

Line Thickness

`pensize(3)`

`pensize(8)`

Turtle Shape

`shape("turtle")`

`shape("arrow")`

Visual Examples

Color Gallery

— red	— purple
— blue	— orange
— green	— pink

Line Thickness

 size 1	 size 8
 size 3	 size 12
 size 6	

Creative Combinations




turtle shape


arrow shape


circle shape

Mix colors, sizes, and shapes for amazing art!

 **Pro Tip:** Use `penup()` to move without drawing, then `pendown()` to start your masterpiece. Experiment with colors and sizes to make it uniquely yours!

Figure 2.3 — Mastering Pen Control — From Basic Lines to Artistic Expression. Your turtle carries a virtual pen that you control completely. Lift it up to move without drawing, put it down to create lines, change colors for mood, adjust thickness for emphasis, and even change your turtle’s appearance. These commands transform simple movement into artistic creation.

The Problem with Manual Repetition

Drawing a hexagon the hard way - notice all the repetition!

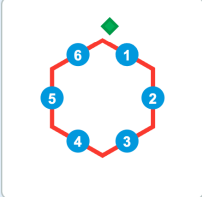
The Hard Way - Typing Everything Out

```

1 import turtle
2 my_turtle = turtle.Turtle()
3 # Drawing a hexagon the hard way
4 my_turtle.forward(100)      ← 1st
5 my_turtle.right(60)         ← 1st
6 my_turtle.forward(100)      ← 2nd
7 my_turtle.right(60)         ← 2nd
8 my_turtle.forward(100)      ← 3rd
9 my_turtle.right(60)         ← 3rd
10 my_turtle.forward(100)     ← 4th
11 my_turtle.right(60)        ← 4th
12 my_turtle.forward(100)     ← 5th
13 my_turtle.right(60)        ← 5th
14 my_turtle.forward(100)     ← 6th
15 my_turtle.right(60)        ← 6th
16 turtle.done()
```

Total Repetition Count:
forward(100): 6 times! right(60): 6 times!

What All This Work Creates



One simple hexagon

Each Step:

- Draw line (forward)
- Turn right 60°
- Repeat 6 times

$360^\circ \div 6 \text{ sides} = 60^\circ \text{ per turn}$

The Problem


SO MUCH TYPING!
15 lines of code

The Pattern:

forward(100)

right(60)

× 6 times

The pattern is obvious!

There Must Be a Better Way!


Coming next:
LOOPS!

💡 Imagine drawing a circle with 360 sides - that would be 720 lines of repetitive code! There has to be a better way...

Figure 2.4— The Problem with Manual Repetition — Why We Need Loops. Drawing one simple hexagon requires typing ‘forward (100)’ six times and ‘right (60)’ six times. That’s 12 lines of nearly identical code! Imagine drawing a circle with 360 sides — you’d need 720 lines of repetitive commands. This tedious repetition is exactly why programming languages provide loops.

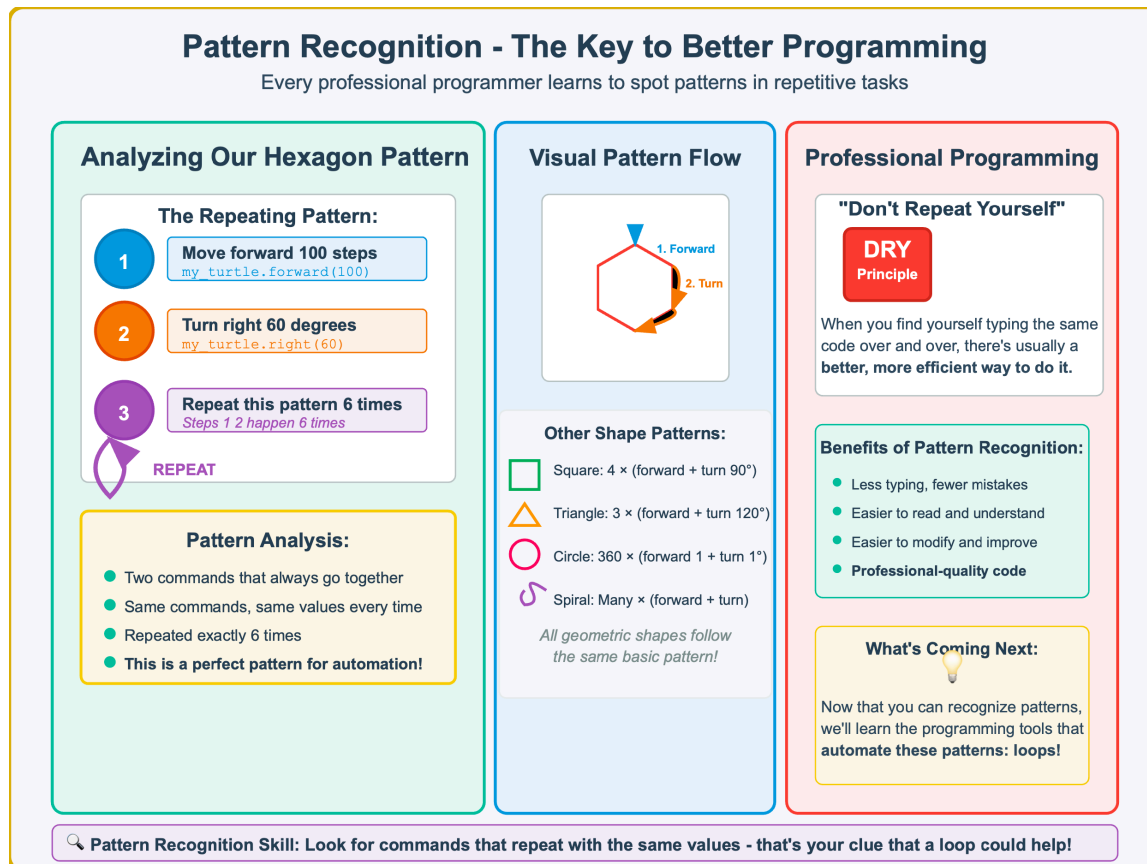


Figure 2.5 — Pattern Recognition — The Key to Better Programming. The difference between a beginner and a professional programmer is the ability to spot patterns. In our hexagon code, the pattern is clear: forward(100), right(60), repeat 6 times. Learning to identify these patterns is your first step toward writing efficient, professional code that follows the DRY principle: Don't Repeat Yourself.

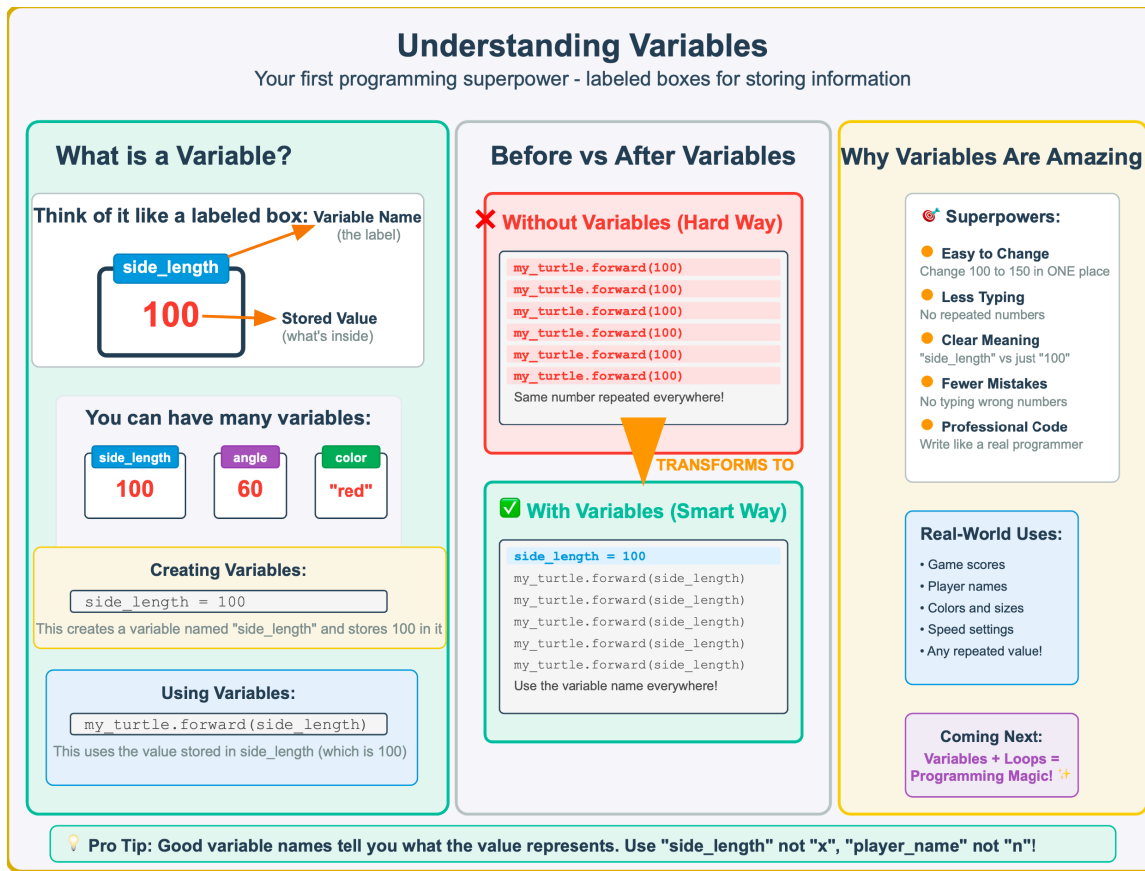
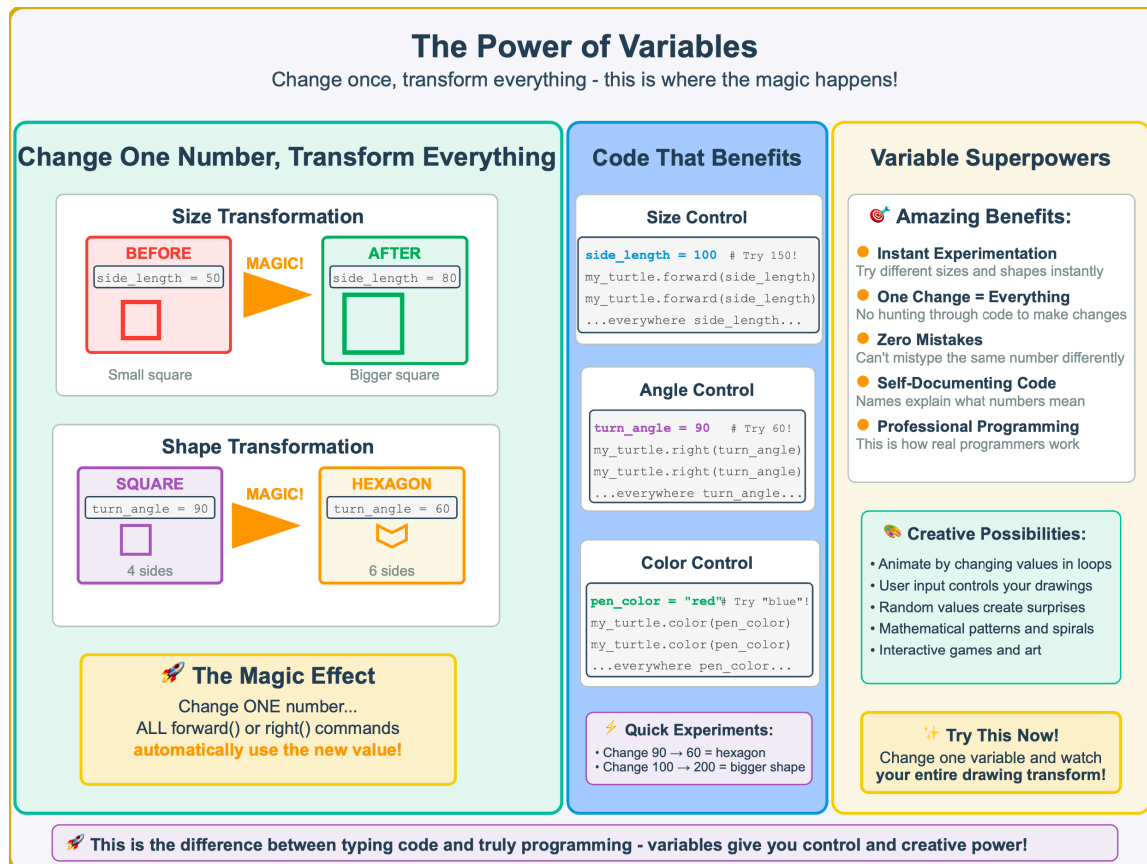


Figure 2.6 — Understanding Variables — Your First Programming Superpower. Variables are like labeled boxes that store information you can use throughout your program. Instead of typing the same number repeatedly, you store it once in a variable and use the variable name everywhere. This makes your code easier to change, understand, and maintain — transforming you from a code typist into a real programmer.



Variable Naming Rules

Writing code that works correctly and makes sense

Essential Rules

Python enforces these requirements

✓ **Rule 1: Valid Characters Only**
Letters (a-z, A-Z), numbers (0-9), underscores (_)

`my_turtle` `player_1`

✗ **Rule 2: Cannot Start with Number**
First character must be a letter or underscore

`2sides` → `sides_2`

⚠ **Rule 3: No Python Keywords**
Avoid reserved words like `if`, `for`, `while`, `def`

`if` `for` `while`

Aa **Rule 4: Case Sensitive**
Capital and lowercase letters create different variables

`myName` ≠ `myname`

Good vs Bad Examples

Learn from common mistakes

✓ **GOOD Variable Names**

`turtle_speed` `player_score`

`side_length` `pen_color`

`max_attempts` `user_name`

Clear, descriptive, follow all rules

✗ **BAD Variable Names**

`2fast` starts with number

`my-turtle` contains hyphen

`for` Python keyword

`x` not descriptive

⚠ **Common Beginner Mistakes:**

- Using spaces: "my turtle" instead of "my_turtle"
- Special characters: "side@length" or "pen\$color"
- Starting with numbers: "1st_player" or "2sides"

Best Practices

Professional naming

✓ **Descriptive**

`turtle_speed`
`player_name`
`circle_radius`

✗ **Poor**

`s`
`x`
`temp`

🔧 **Style Guide:**

- Use snake_case
- Be specific
- Avoid abbreviations
- Use full words
- Stay consistent

💡 **Pro Tip:**
Variable names are documentation!

Remember: Good variable names make your code self-explaining. Choose names that tell the story of what your program does!

Figure 2.8 — Variable Naming Rules — Writing Code That Works and Makes Sense. Python has specific rules for variable names that you must follow, plus best practices that make your code professional and readable. Master these naming conventions and you'll write code that not only works correctly but also communicates clearly to anyone reading it.

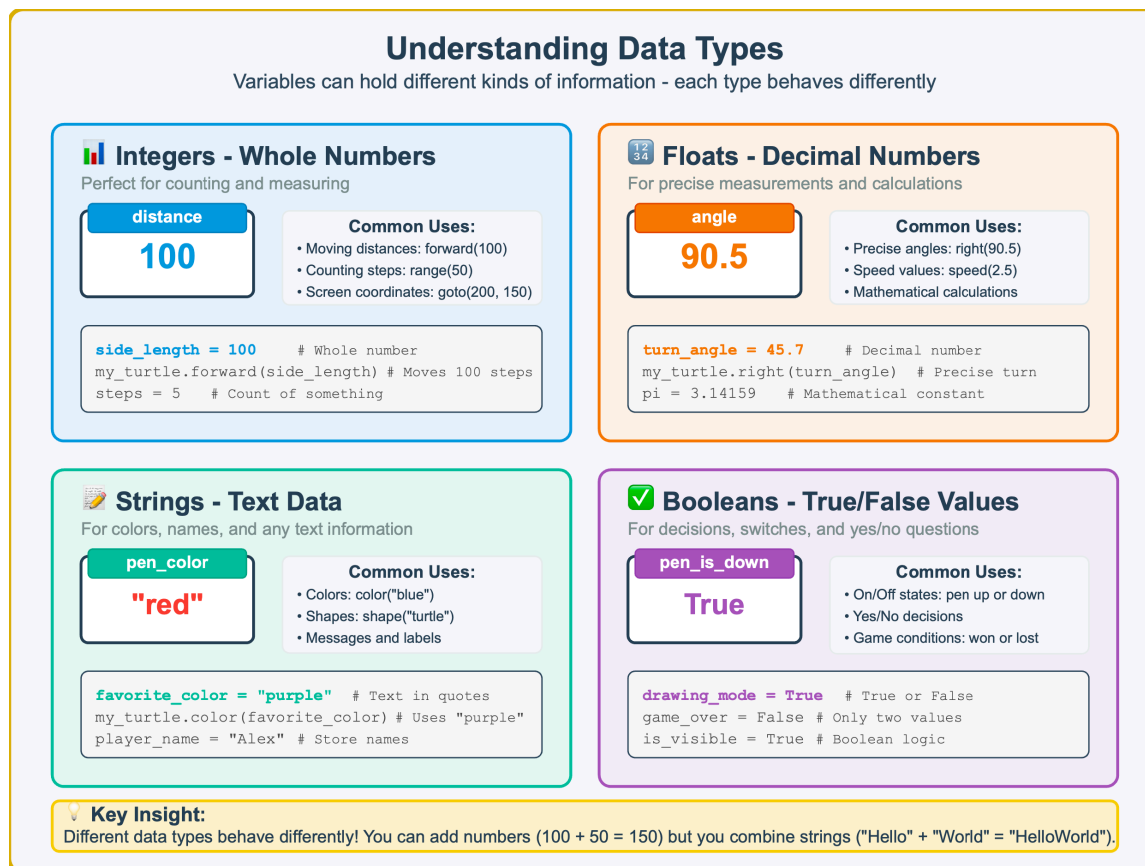


Figure 2.9 — Understanding Data Types — Different Kinds of Information in Variables. Just as different boxes hold different kinds of objects, variables can hold different types of data. Integers store whole numbers for counting and measuring, floats store decimal numbers for precision, strings store text for colors and names, and booleans store True/False for decisions. Master these four types and you can represent any information your program needs.

Variables and Math Operations

Making variables work for you - from storage to computation

Basic Math Operations

The mathematical building blocks of programming

+ **Addition**
Combine values together
`5 + 3 = 8` Numbers combine

- **Subtraction**
Find the difference
`10 - 4 = 6` Reduce by amount

× **Multiplication**
Repeated addition
`6 * 7 = 42` Scale up values

÷ **Division**
Split into equal parts
`15 / 3 = 5` Break into pieces

****** **Power (Exponent)**
Multiply by itself
`2 ** 3 = 8` 2×2×2

Variables in Calculations

Where variables become truly powerful

Setting Up Variables:

```
side_length = 100
sides = 6
```

Addition with Variables

```
total = side_length + 50
Result: total = 150 (because 100 + 50 = 150)
```

Multiplication with Variables

```
perimeter = side_length * sides
Result: perimeter = 600 (because 100 * 6 = 600)
```

Complex Calculations

```
angle = 360 / sides
Result: angle = 60 (because 360 / 6 = 60)
```

The Magic:
Change 'sides = 6' to 'sides = 8' and
everything recalculates automatically!

Turtle Applications

Real-world uses

Size Scaling

```
size * 2
Double the size
size / 2
Half the size
```

Smart Angles

```
360 / sides
Perfect polygons
angle / 2
Gentle curves
```

Patterns

```
step * 2
Growing spirals
x + y * 3
Complex math
```

Get Creative!

- Scaling shapes
- Color mixing
- Speed control
- Dynamic art

Power Unlock: Math + Variables = Programs that think, calculate, and adapt! Your code becomes intelligent and flexible.

Figure 2.10 — Variables and Math Operations — Making Variables Work for You. Variables become truly powerful when you use them in calculations. Instead of just storing single values, you can add, subtract, multiply, and divide variables to create new values dynamically. This transforms your programs from static instructions into flexible, mathematical tools that adapt and calculate.

Introducing For Loops

The end of repetitive typing - Python's way to automate repetition

The Problem Loops Solve

❌ THE OLD WAY (Tedious!)

```
my_turtle.forward(100)
my_turtle.forward(100)
my_turtle.forward(100)
my_turtle.forward(100)
...and on and on...
```

TRANSFORMS TO

✅ THE NEW WAY (Elegant!)

```
for i in range(100):
    my_turtle.forward(100)
# Just 2 lines!
```

🦸 Loop Superpowers:

- Less typing = fewer mistakes
- Easy to change: modify one number
- Works with any number of repetitions
- This is how real programmers work!

For Loop Syntax

Every part has a purpose

The Complete Structure:

```
for i in range( 6 ) :
    my_turtle.forward(100)
```

Component Breakdown:

- for...in** Python's loop command
- i** Loop counter (you can name it anything)
- range(6)** Repeat 6 times
- :** Required! Starts the loop block
- 4 spaces →** Code to repeat

⚠ Critical Points:

- Don't forget the colon (:) at the end
- Indentation must be exactly 4 spaces
- Every line in the loop needs indentation

Examples

Simple Print

```
for i in range(3):
    print("Hi")
```

Prints "Hi" 3 times

Draw Square

```
for i in range(4):
    forward(100)
    right(90)
```

 Perfect square!

With Variables

```
sides = 6
for i in range(sides):
    forward(50)
    right(60)
```

 Hexagon!

🚀 Try It!

Change the numbers and see what happens!

🎉 **Congratulations!** You've just learned the programming concept that separates beginners from real programmers. Welcome to automation!

Figure 2.11 — Introducing For Loops — The End of Repetitive Typing. Remember typing ‘forward(100)’ six times for a hexagon? Those days are over! For loops let you write ‘for i in range(6):’ once, then ‘forward(100)’ once, and Python repeats it six times automatically. This is the programming breakthrough that transforms tedious typing into elegant automation — the moment you become a real programmer.

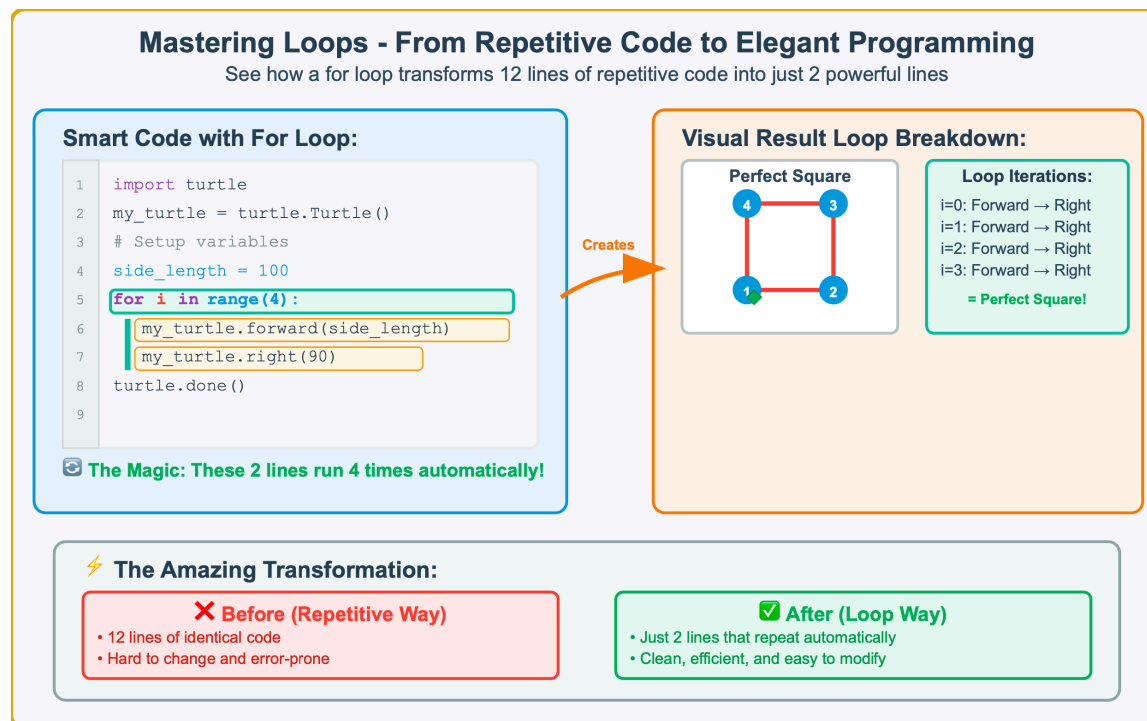


Figure 2.12 — Mastering Loops — From Repetitive Code to Elegant Programming. This comparison demonstrates how a for loop transforms 12 lines of repetitive code into just 2 lines that execute automatically. The visual breakdown shows how `range(4)` creates exactly 4 loop iterations, each drawing one side of the square, illustrating the power and elegance of programming loops

Understanding Python Indentation

How spaces at the beginning of lines tell Python which code belongs inside loops

⚠ CRITICAL CONCEPT: Indentation (spaces) is not just for looks - it controls how your program runs!

Python Code Example:

```
1 0 SPACES for i in range(3): ← Loop header
2 4 SPACES print("Inside the loop") ← 4 spaces in
3 4 SPACES print("Also inside") ← 4 spaces in
4 print("Outside the loop") ← No spaces
5
```

🔑 The Rule:

- Lines with the SAME indentation belong together
- Use exactly 4 spaces (or 1 Tab) for each level

Program Output:

Console Output:

```
Inside the loop
Also inside
Inside the loop
Also inside
Inside the loop
Also inside
Outside the loop
```

📌 Notice: Indented lines run 3 times, unindented line runs once

💡 Pro Tips for Perfect Indentation:

✅ DO THIS

- Use 4 spaces consistently (or set Tab = 4 spaces)
- All lines at same level = same indentation

❌ AVOID THIS

- Mixing spaces and tabs (causes errors!)
- Inconsistent spacing (2 spaces, then 3, then 5...)

Figure 2.13 — Understanding Python Indentation — How Spaces Control Program Flow. This example demonstrates Python’s crucial indentation rule: lines with the same indentation level belong to the same code block. The 4—space indented lines run inside the loop (3 times), while the unindented line runs outside the loop (once). Proper indentation is not optional in Python — it determines how your program executes.

The Magic of Loop Flexibility

Change just one number in range() and create completely different shapes!

Complete Code Example:

```

1 import turtle
2 my_turtle = turtle.Turtle()
3 my_turtle.speed(6)
4 # Triangle (3 sides)
5 for i in range( 3 ):
6     my_turtle.forward(100)
7     my_turtle.right(120) 360÷3
8 # Move to new position
9 my_turtle.goto(150, 0)
10 for i in range( 6 ):
11     my_turtle.forward(80)
12     my_turtle.right(60) 360÷6

```

Only the range() number and turn angle change!

Visual Results Pattern:

Same Code Pattern, Different Shapes



Triangle
range(3)



Square
range(4)



Hexagon
range(6)

The Magic Formula:

Turn Angle = 360° ÷ Number of Sides

Try These Combinations:

range(5) + 72° = Pentagon	range(8) + 45° = Octagon
range(10) + 36° = Decagon	range(12) + 30° = 12-gon

The higher the number, the more it looks like a circle!

The Beautiful Truth About Programming:

✨ Small Changes

Change just one number
Create totally new results

🔄 Same Pattern

The loop structure stays identical
Only parameters change

∞ Infinite Possibilities

Any polygon is possible
Mathematics meets creativity

Figure 2.14 — The Magic of Loop Flexibility — One Pattern, Many Shapes. This example demonstrates the incredible power of for loops: by changing just one number in range(), the same code pattern creates completely different geometric shapes. The mathematical relationship (Turn Angle = 360° ÷ Number of Sides) shows how programming connects to fundamental geometric principles, opening infinite creative possibilities.

Beyond Numbers - Looping Through Lists of Data

Discover how for loops can work with lists of words, colors, or any data you want!

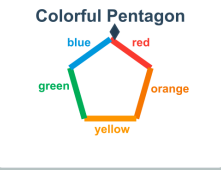
Colorful Drawing with Lists:

```

1 import turtle
2 my_turtle = turtle.Turtle()
3 # Create a list of colors
4 colors = ["red", "orange", "yellow",
5           "green", "blue"]
6
7 for color in colors:
8     my_turtle.color(color)
9     my_turtle.forward(50)
10    my_turtle.right(72) # 360÷5
11
12 turtle.done()
```

NEW: Instead of range(5), we use a list of actual values!

Visual Result Process:



Colorful Pentagon

Loop Process:

Step 1: color = "red"

Step 2: color = "orange"

Step 3: color = "yellow"

Step 4: color = "green"

Step 5: color = "blue"

Each color from the list!

The Loop Goes Through Each Item:

red → orange → yellow → green → blue

Two Ways to Loop:

- With numbers: for i in range(5)
- With lists: for color in colors

Key Concepts - Lists Make Loops More Powerful:

Lists Store Multiple Items

- Square brackets: [item1, item2, item3]
- Can hold words, numbers, anything!
- Colors: ["red", "blue", "green"]

"for item in list" Pattern

- Variable gets each list item automatically
- No counting or range() needed
- Python handles everything for you!

Real-World Applications

- Process student names from a class list
- Draw with different colors or sizes
- Handle any collection of data!

Figure 2.15 — Beyond Numbers — Looping Through Lists of Data. This example introduces a powerful expansion of for loops: instead of looping through numbers with range(), you can loop through any list of data. The ‘for item in list’ pattern automatically assigns each list element to the variable, making it easy to process collections of colors, names, or any data. This opens up endless possibilities for data—driven programming.

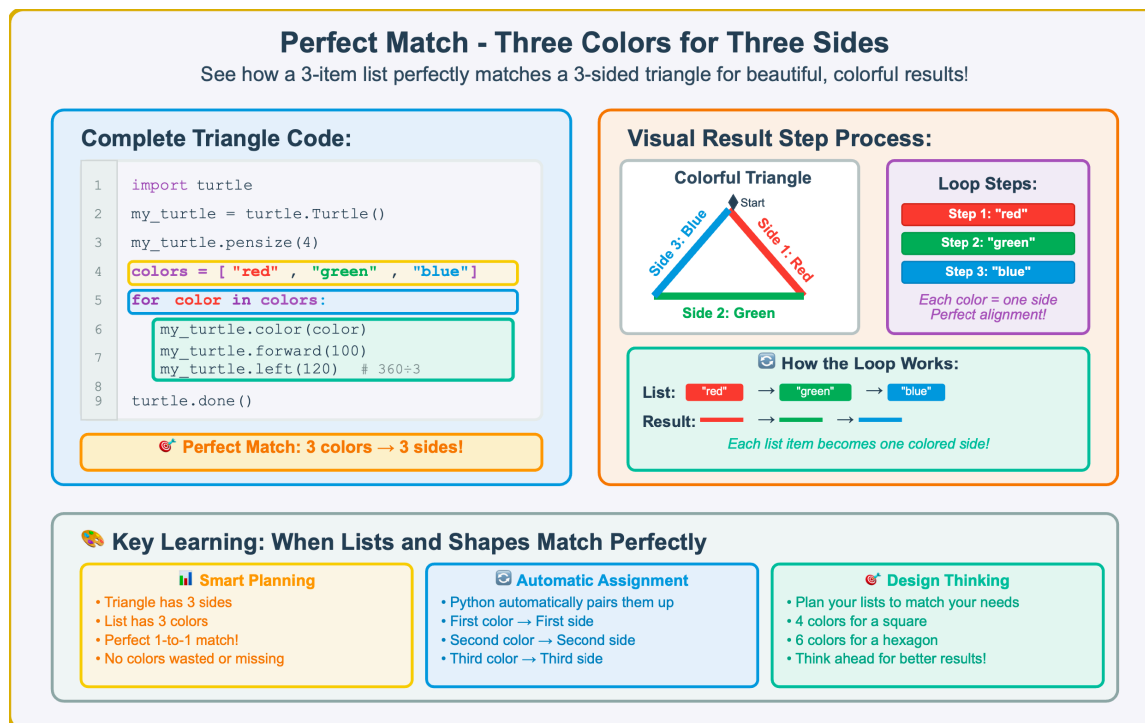


Figure 2.16 — Perfect Match — Three Colors for Three Sides. This example demonstrates the beauty of thoughtful programming: a 3—item color list perfectly matches a 3—sided triangle. Each loop iteration automatically assigns the next color to the next side, creating a seamless flow from data to visual result. This introduces the important concept of planning data structures to match your program’s needs.

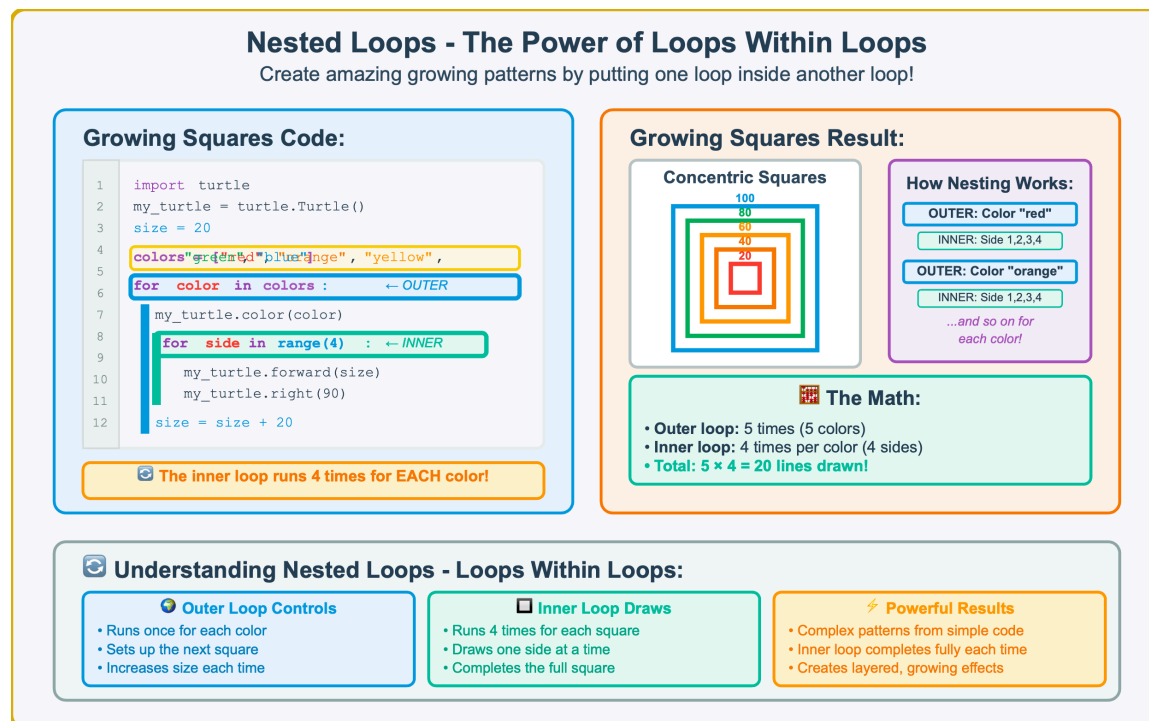


Figure 2.17 — Nested Loops — The Power of Loops Within Loops. This example demonstrates nested loops, where one loop runs inside another. The outer loop cycles through 5 colors, and for each color, the inner loop draws 4 sides to create a complete square. This creates 5 concentric squares (5 outer $\times$ 4 inner = 20 total lines), each larger and differently colored than the last, showing how nested loops can create complex, beautiful patterns.

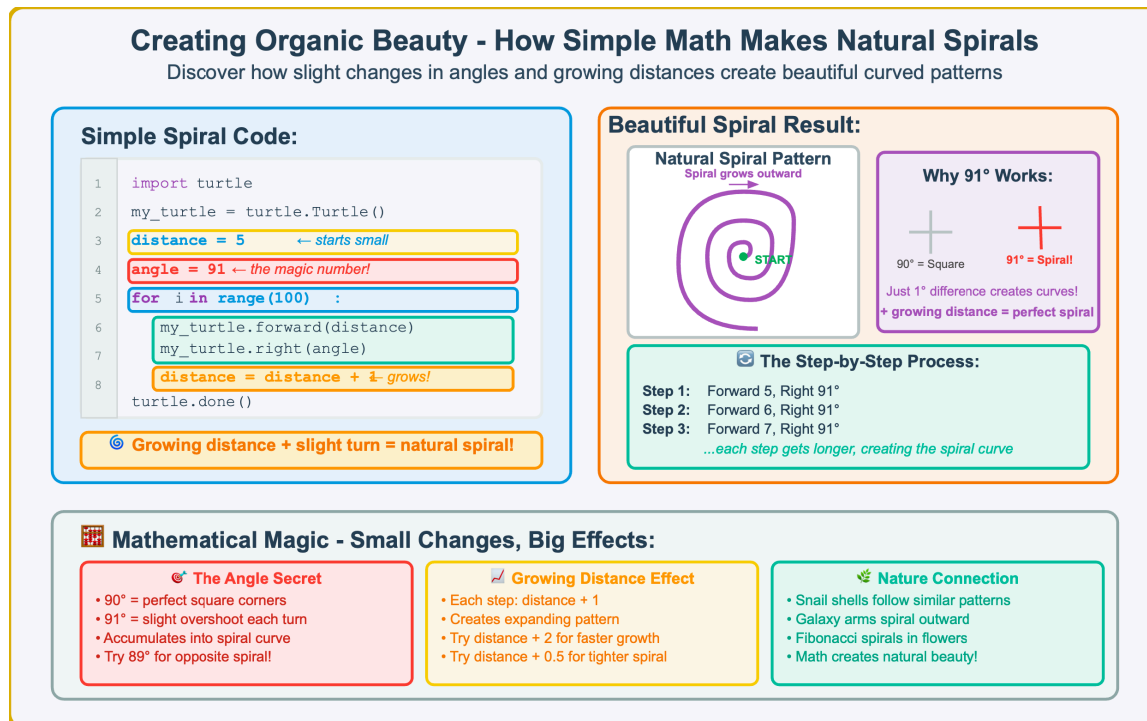


Figure 2.18 — Creating Organic Beauty — How Simple Math Makes Natural Spirals. This example demonstrates how small mathematical changes create dramatic visual effects: a 91° turn (just 1° more than a square's 90°) combined with gradually increasing distances produces a natural spiral pattern. This illustrates how programming can recreate the mathematical principles found in nature, from snail shells to galaxy formations.

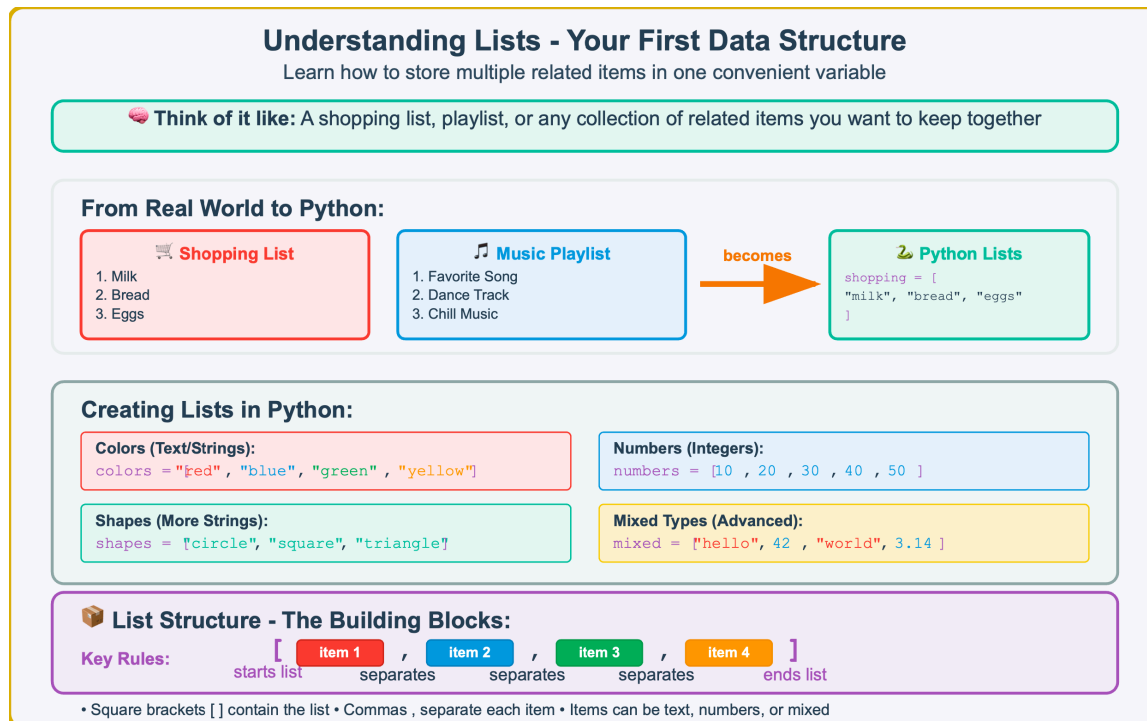


Figure 2.19 — Understanding Lists — Your First Data Structure. Lists allow you to store multiple related items in a single variable, just like real—world collections such as shopping lists or playlists. Python lists use square brackets [] to contain items and commas to separate them. Lists can hold text (strings), numbers, or even mixed data types, making them incredibly versatile for organizing and processing related information.

The Perfect Partnership - Lists and Loops Working Together

See how lists provide data for loops to process - the foundation of data-driven programming!

? Challenge: Draw squares of different sizes (50, 75, 100, 125, 150) without copying code
💡 Smart Solution: Store sizes in a list, let the loop handle the rest!

Data-Driven Solution:

```
1 import turtle
2 my_turtle = turtle.Turtle()
3 sizes = [50, 75, 100, 125, 150]
4 for size in sizes: ← processes list
5     for side in range(4): ← draws square
6         my_turtle.forward(size)
7         my_turtle.right(90)
8     my_turtle.forward(size + 20)
9     turtle.done()
```

📊 List drives the loop - each size becomes a square!

Progressive Squares Result:

Each List Item = One Square



How the Partnership Works:

List provides data: 50 → 75 → 100 ...

Loop processes each: size=50 → draws 50×50 square
size=75 → draws 75×75 square

Why This Partnership is Powerful:

- 📊 Data-Driven**
 - Change the list, change the result
 - Add [200, 250] for bigger squares!
- 📧 No Code Duplication**
 - Write the drawing code once
 - Loop handles all repetitions
- 🔄 Flexible Scalable**
 - Works with any number of items
 - Foundation for processing any data

Figure 2.20 — The Perfect Partnership — Lists and Loops Working Together. This example demonstrates the fundamental programming pattern of data—driven loops: a list provides the data ([50, 75, 100, 125, 150]), and a for loop processes each item automatically. The outer loop iterates through sizes while the inner loop draws each square. This eliminates code duplication and makes programs flexible — changing the list instantly changes the output.

Pinpoint Access - Getting Specific Items from Lists

Learn how to access any item in a list using position numbers (starting from 0!)

💡 **Key Concept:** Each item in a list has a position number (index) starting from 0, not 1!

How List Indexing Works:

```
colors = [ "red" , "blue" , "green" , "yellow" ]
```

Index 0
↑ FIRST

Index 1
↑ SECOND

Index 2
↑ THIRD

Index 3
↑ FOURTH

⚠️ **CRITICAL:** Python starts counting from 0, not 1! First item = Index 0

Accessing Specific Items:

Get FIRST item (Index 0):
`print(colors[0])` # Prints "red"

Get SECOND item (Index 1):
`print(colors[1])` # Prints "blue"

Get THIRD item (Index 2):
`print(colors[2])` # Prints "green"

Syntax: `list_name[index_number]`

What Gets Printed:

Console Output:

```
red
blue
green
```

Practical Use:

```
my_turtle.color(
    colors[0])
```

Always uses red for first shape

Index → Item Mapping:

0	→ "red"	1	→ "blue"	2	→ "green"
---	---------	---	----------	---	-----------

🔑 **Remember:** Index 0 = First item, Index 1 = Second item, Index 2 = Third item... This gives you precise control over your data!

Figure 2.21 — Pinpoint Access — Getting Specific Items from Lists. List indexing allows you to access any specific item using its position number in square brackets. Python uses zero—based indexing, meaning the first item is at index 0, the second at index 1, and so on. The syntax `list_name[index_number]` gives you precise control over which data element to use, enabling targeted access to specific information in your programs.



Put It All Together - Rainbow Polygon Challenge

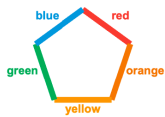
Apply everything you've learned: lists, loops, colors, and angles!



Your Mission: Create a polygon where each side is a different color - combine lists, loops, and math!
 This challenge combines everything from this chapter into one beautiful program.

What You're Building:

Rainbow Pentagon Example



Skills Used:

- ✓ Lists
- ✓ For loops
- ✓ Colors
- ✓ Angles
- ✓ Math
- ✓ Turtle graphics

Starter Code Template:

```

1  import turtle
2  my_turtle = turtle.Turtle()
3  # 1. Create your color list here
4  # 2. Calculate the angle (360 ÷ sides)
5  # 3. Write your for loop here
6  # 4. In loop: set color, draw side, turn
7
8  turtle.done()

```

Step-by-Step Hints:

Step 1: colors = ["red", "orange", "yellow", "green", "blue"]

Step 2: angle = 360 ÷ 5 = 72 (for pentagon)

Your Requirements:

- Create a color list (5+ colors)
- Use a for loop with colors
- Calculate correct angle
- Each side different color

Once You Succeed - Try These Extensions:

More Colors

- Add more colors to your list
- Try 8 colors for an octagon!

Different Sizes

- Make each side a different length
- Create a list of sizes too!

Patterns

- Draw multiple polygons
- Make a rainbow flower pattern!

Figure 2.22 — Put It All Together — Rainbow Polygon Challenge. This hands-on challenge combines all the concepts from this chapter: creating lists, using for loops, applying colors, and calculating angles. Students build a polygon where each side is a different color, demonstrating their mastery of data-driven programming. The challenge includes starter code, step-by-step hints, and extension ideas for continued learning.

Advanced Challenge - Shrinking Circles with Dynamic Variables

Learn to modify variables inside loops - a powerful programming technique!

New Challenge: Create circles that get smaller and smaller using a variable that changes inside the loop!
 This introduces a key concept: **modifying variables during loop execution**.

What You're Building:

Shrinking Circle Pattern

Each circle gets smaller!

Challenge Requirements:

- Start: radius = 100
- Decrease by 10 each time

Code Template New Techniques:

```

1  import turtle
2  my_turtle = turtle.Turtle()
3  radius = 100  ← starting value
4  # Your loop here
5  # Draw circle(radius)
6  # Change radius value
7  turtle.done()

```

Two Different Approaches:

While Loop

```

while radius > 0:
    # draw shrink

```

For Loop

```

for i in range(10):
    # draw shrink

```

Step-by-Step Instructions:

Step 1: Setup

- Create variable: radius = 100
- Create color list (optional)

Step 2: Loop

- Check: while radius > 0
- Draw: my_turtle.circle(radius)

Step 3: Modify

- Shrink: radius = radius - 10
- Change color (optional)

Figure 2.23 — Advanced Challenge — Shrinking Circles with Dynamic Variables. This challenge introduces a powerful programming concept: modifying variables inside loops. Students create concentric circles that get progressively smaller by changing the radius variable during each loop iteration. This technique ($\text{radius} = \text{radius} - 10$) demonstrates how variables can evolve during program execution, opening up new possibilities for dynamic, changing patterns.



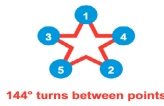
Creative Challenge - Building Your Star Constellation

Master complex geometry and create beautiful patterns with five-pointed stars!

 **Creative Challenge:** Draw multiple five-pointed stars in different positions, sizes, and colors!
 This combines movement, colors, loops, and complex geometry - the ultimate test of your skills!

Understanding Star Geometry:

Five-Pointed Star



144° turns between points

The Star Secret:

Magic Number: 144°

Why 144°?

- 5 points in a star
- Skip every other point
- $360^\circ \div 5 = 72^\circ$
- $72^\circ \times 2 = 144^\circ$!

Basic Star Pattern:

```

for i in range(5) :
    my_turtle.forward(50)
    my_turtle.right(144) ← The magic!
```

Your Creative Mission:

Your Star Constellation



 **Your Requirements:**

1 Draw 4-6 stars

Use the 144° pattern
5 points each
for i in range(5)

2 Different colors

my_turtle.color()
between each star
Use bright colors!

3 Move positions

my_turtle.goto(x,y)
Spread them out
Make a pattern!



Starter Code Template:

```

import turtle
my_turtle = turtle.Turtle()
# Your constellation code here!
```

 **Pro Tips for Success:**

- Start with one star, then copy the pattern
- Use my_turtle.penup() and pendown() for clean movement

Figure 2.24 — Creative Challenge — Building Your Star Constellation. This capstone challenge introduces students to complex geometric shapes while encouraging creative expression. Students learn that five—pointed stars require 144° turns (derived from $360^\circ \div 5 \times 2$), combining mathematical reasoning with artistic creation. The challenge integrates all chapter concepts: loops, colors, movement, and geometry to create beautiful star constellations.



Code-Along Project - Creating Natural Beauty with Curved Petals

Learn advanced turtle techniques to create organic, nature-inspired patterns!


Code-Along Project: Follow step-by-step to create beautiful flower petals using curved lines!
New technique: Using `circle(size, angle)` to create partial circles and natural curves.

Complete Flower Code:

```

1  import turtle
2  my_turtle = turtle.Turtle()
3  my_turtle.speed(9)
4  my_turtle.color("pink")
5  number_of_petals = 8      ← adjustable!
6  for petal in range(number_of_petals):
7      my_turtle.circle(60, 60) ← NEW!
8      my_turtle.left(120)
9
10
11  my_turtle.left(360 / number_of_petals)
12  turtle.done()
    
```

🔑 Key: `circle(size, degrees)` draws partial circles!

Beautiful Flower Result:

8-Petal Pink Flower



Natural curved petals!

New Technique:
`circle(size, angle)`

- size = radius
- angle = how much of circle to draw

`circle(60, 60):`
 = 1/6 of a circle
 = perfect petal curve!


How Each Petal Forms:

- Step 1:** Draw curve: `circle(60, 60)`
- Step 2:** Turn: `left(120)` to complete petal
- Step 3:** Position: `left(45)` for next petal



What You'll Learn from This Project:


Organic Shapes

- `circle(size, angle)` for curves
- Natural vs geometric patterns


Advanced Math

- $360^\circ \div \text{petals} = \text{spacing}$
- Partial circles and arcs


Pattern Repetition

- Complex shapes from simple loops
- Variables for easy customization

Figure 2.25 — Code—Along Project — Creating Natural Beauty with Curved Petals. This guided project introduces advanced turtle graphics techniques, specifically using `circle(size, angle)` to create partial circles for natural, organic shapes. Students learn to create flower petals by combining curved lines with precise mathematical spacing ($360^\circ \div \text{number_of_petals}$), bridging the gap between geometric programming and artistic, nature—inspired design.

Code Along: Colorful Spiral Square

Combining loops, colors, and creativity for stunning visual effects!

Complete Code:

```
import turtle
my_turtle = turtle.Turtle()
my_turtle.speed(9)

# Spiral parameters
size = 10
colors = [ "red", "orange",
           "yellow", "green",
           "blue", "purple"]

# Draw spiral of squares
for i in range(36):
    # Choose color
    color_index = i % len(colors)
    my_turtle.color(colors[color_index])

    # Draw square
    for side in range(4):
        my_turtle.forward(size)
        my_turtle.right(90)
    my_turtle.right(10) # Next position
```

Visual Result:



36 colorful squares in perfect spiral!

How The Magic Works:

- **36 squares:** Main loop runs 36 times
- **Color cycling:** % operator cycles through 6 colors
- **Spiral effect:** Each square rotated 10° more
- **Nested loops:** Outer for squares, inner for sides

🔑 Key Programming Concepts:

Lists Indexing • Modulo Operator (%) • Nested Loops • Variable Progression

Figure 2.26 — Code Along — Creating a Colorful Spiral Square. This intermediate project demonstrates how combining nested loops, list indexing, and the modulo operator creates complex visual patterns. Students learn color cycling, spiral mathematics, and how small incremental changes (10° rotation) produce dramatic visual effects. This project bridges basic programming concepts with advanced graphics programming.

Code Along: Circle Art Creation

Overlapping circles create flower-like patterns through mathematical symmetry!

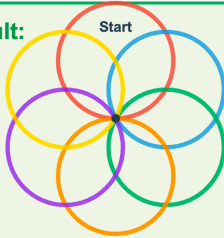
Complete Code:

```
import turtle
my_turtle = turtle.Turtle()
my_turtle.speed(9)

# Circle art parameters
radius = 50
colors = ["red", "blue", "green", "orange", "purple", "yellow"]

# Draw overlapping circles
for i in range(6):
    my_turtle.color(colors[i])
    my_turtle.circle(radius)
    my_turtle.right(60) # Turn
turtle.done()
```

Visual Result:



Perfect symmetry creates natural beauty!

The Pattern:

- 6 circles, 6 colors
- Each circle: radius 50
- Turn 60° between circles
- Same center point

The Math:

$360^\circ \div 6 \text{ circles} = 60^\circ$

Perfect spacing creates symmetry

Overlapping = Beauty!

Key Programming Concepts:

List Indexing • Circle Method Mathematical Symmetry • Color Patterns

Pro Tip:

Same starting point + rotation = flower-like patterns

Figure 2.27 — Code Along — Circle Art with Mathematical Symmetry. This project beautifully demonstrates how mathematical principles create artistic patterns. Students learn that 360° divided by 6 circles equals 60° per rotation, creating perfect symmetry. The overlapping circles form natural flower—like patterns, showing how programming connects mathematics, geometry, and visual art.

Programming Challenge: Build Your Dream House

Use variables, loops, and creativity to construct a house with code!

Your Challenge:

Requirements:

- Use **variables** for house dimensions and colors
- Use a **loop** to draw the square house base
- Add a **roof** using precise positioning
- Include **door and windows** with details
- Use **fill colors** to make it beautiful

Code Structure Hints:

```
# House parameters
house_size = 100
roof_color = "red"

# Draw house base with loop
for side in range(4):
    my_turtle.forward(house_size)
```

Your Goal:



Build this... or make it even better!

Bonus Ideas:

- Add a chimney with smoke
- Draw a fence around the house
- Create multiple houses
- Add a garden or trees

Pro Tips:

- Plan your drawing order
- Use `goto()` for positioning
- Test one part at a time
- Have fun and be creative!

What You'll Practice:

Variables make code flexible and reusable • **Loops** reduce repetitive coding • **Positioning** with `goto()` for precise placement
Colors Fill bring drawings to life • **Planning** breaks complex projects into simple steps

Figure 2.28 — Programming Challenge — Build Your Dream House. This integrative challenge allows students to apply all Chapter 2 concepts in a creative, personal project. Students use variables for flexibility, loops for efficiency, and precise positioning to construct their own house design. The challenge encourages both technical skill development and creative expression, preparing students for independent programming projects.

Mathematical Challenge: Rainbow Polygon Gallery

Create a colorful showcase of geometric shapes using mathematical formulas!

Your Mathematical Challenge:

Requirements:

- Draw shapes from 3 to 8 sides (triangle to octagon)
- Calculate angles using formula: $360^\circ \div \text{sides}$
- Use different colors for each polygon
- Position shapes in an organized gallery layout
- Use nested loops for efficient drawing

The Mathematical Key:

Interior Angle = $360^\circ \div \text{number_of_sides}$

Triangle (3): $360^\circ \div 3 = 120^\circ$ Square (4): $360^\circ \div 4 = 90^\circ$

```
for sides in range(3, 9):
    angle = 360 / sides # Math!
```

Your Geometric Gallery:



Triangle
3 sides



Square
4 sides



Pentagon
5 sides



Hexagon
6 sides



Heptagon
7 sides



Octagon
8 sides

Programming Tips:

- Use `penup()` to move
- Test one shape first
- Plan your layout
- Use color lists

Bonus Ideas:

- Fill shapes with colors
- Arrange in a circle
- Add shape labels
- Create size variations

Mathematical Programming Concepts:

Mathematical Formulas in code • **Nested Loops** for efficiency • **Geometric Understanding** through programming
Color Cycling with modulo operator • **Positioning** and layout planning • **Pattern Recognition** in mathematics

Figure 2.29 — Mathematical Challenge — Rainbow Polygon Gallery. This sophisticated challenge demonstrates the powerful intersection of mathematics and programming. Students apply the geometric formula ($360^\circ \div \text{sides}$) to calculate interior angles for polygons ranging from triangles to octagons. The project reinforces mathematical concepts through visual programming, showing how geometric principles translate directly into code logic and artistic expression.

Mindful Coding Challenge: Create a Digital Mandala

Combine programming with meditative art to create beautiful circular patterns!

🧘 Your Mandala Challenge:

What is a Mandala?

A mandala is a spiritual and ritual symbol representing the universe, often featuring radial symmetry and patterns.

Programming Requirements:

- Draw 36 decorative shapes ($360^\circ \div 36 = 10^\circ$)
- Use quarter circles (90° arcs) for petal shapes
- Rotate 10° between each shape for symmetry
- Cycle through colors using modulo operator
- Create radial symmetry from center point

Key Code Pattern:

```
for i in range(36):
    my_turtle.circle(size, 90# Arc
    my_turtle.right(10) # Rotate
```

🌸 Your Digital Mandala:



Perfect symmetry through code! ✨

💡 Coding Tips:

- Start with one petal
- Use `circle(size, 90)`
- Test rotation angles
- Embrace imperfection

🧘 Mindful Coding:

- Focus on the process
- Notice patterns emerge
- Find joy in repetition
- Code becomes meditation

🌸 Art Meets Science:

Radial Symmetry through mathematical precision • Repetitive Patterns create meditative flow • Color Theory meets algorithm design
Programming becomes a **spiritual practice** where logical structure creates transcendent beauty and mindful focus

Figure 2.30 — Mindful Coding Challenge — Create a Digital Mandala. This unique capstone project demonstrates how programming can become a contemplative practice. Students create beautiful radial patterns using mathematical precision ($360^\circ \div 36 = 10^\circ$ rotation), discovering how repetitive coding can achieve a meditative flow state. The challenge bridges ancient spiritual art traditions with modern programming, showing that code can be both logical and transcendent.